



COPILOTTRAINING
TECHNICAL BRIEFING

Agentic Lifecycle Orchestration

| Evidence-Gated Handoffs from Issue to Pull Request

Make every transition inspectable, authorized, recoverable, and owned

CopilotTraining · Practitioner Tech Talk



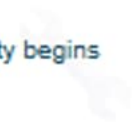
"Which recurring repository judgments belong in workflows?"

Qualify each handoff by evidence, mutation boundaries, and authority.

A candidate lifecycle earns trust only after target-repository validation.

Developer

See where implementation authority begins and stops



Team Lead

Assign owners to evidence-gated handoffs



Platform Engineer

Inspect compilation, permissions, and safe outputs



4 workflow sources

Candidate phases from intake through review

1 exact approval

A fresh plan advances only at an authorized gate

0 runtime claims

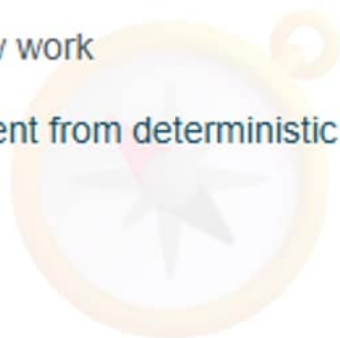
Transitions remain expected until repository validation

PART 1

Select Judgments

Choose bounded workflow work

Separate recurring judgment from deterministic automation



PART 2

Visible State

Expose evidence and ownership

Narrate issue #482 through inspectable candidate states

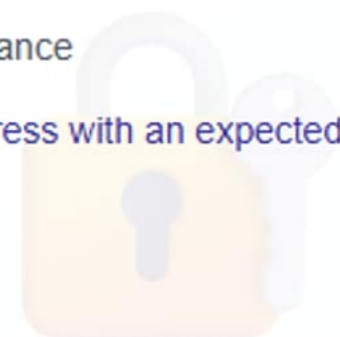


PART 3

Preserve Authority

Gate mutation and acceptance

Compare authorized progress with an expected no-op

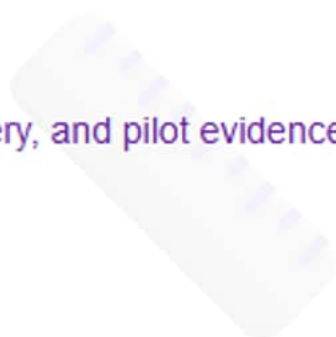


PART 4

Measure and Recover

Keep stops in the system

Define local timing, recovery, and pilot evidence



Select Workflow-Owned Judgments

Qualify each candidate by trigger, permissions, mutation boundaries, compilation, and safe outputs.



Trigger

Name the bounded event



Authority

Constrain reads and writes



Compile

Inspect generated permissions

Candidate lifecycle

↳ 4 sources · 0 compiled claims

One Decision per Workflow



Intake

Is one issue specific, non-duplicative, and routable enough to plan?



Planning

Is the proposed scope executable, testable, reversible, and owned?



Coding

Was only the latest authorized plan implemented in one repository?



Review

Is the evidence ready for deterministic checks and human acceptance?

A Workflow Must Explain Its Authority

Recurrence alone is not enough

Input	Name the event and repository evidence trusted by this phase	bounded
Decision	Assign one recurring judgment that cannot be a deterministic step	owned
Evidence	Emit a durable comment, label, pull request, or review result	visible
Stop	Stop on missing context, failed checks, drift, or absent authority	explicit
Recovery	Name the person or team able to supply the next evidence	routable

Deterministic builds stay in Actions; unbounded judgment stays human-led

Source Intent Is Not Executable Proof

target repository

1. Copy four .md sources to .github/workflows/
2. Run: `gh aw compile`
3. Inspect four generated .lock.yml files
4. Commit source and lock files together

Candidate source

Markdown records triggers, read access, instructions, and declared safe outputs

Generated permissions

Lock files expose the actual jobs and write-capable handlers for review

Runtime gate

Only a bounded target-repository run can validate transition behavior

Make the State Machine Visible

Follow issue #482 from intake evidence to a bounded plan and stop at the approval gate.



Intake

Evidence qualifies entry



Plan

Scope and rollback stay visible



Stop

Approval blocks mutation

`Narrated candidate walkthrough`

`↳ Issue #482 · evidence before transition`

A Label Shows State; Evidence Permits Motion

Stop labels take precedence over accumulated success milestones

Intake

Issue input produces intake evidence or a visible stop

triaged

Planning

Intake evidence produces a bounded plan or a visible stop

planned

Coding

Fresh plan plus authorized approval produces one draft PR

in-review

Review

Plan, approval, diff, and checks produce advisory evidence

reviewed

State + required evidence + next-owner authority must agree

Issue #482: Intake Produces Planning Evidence

A walkthrough of intended artifacts, not an executed workflow

```
$ candidate walkthrough: issue #482
```

```
> "Issue arrives with acceptance criteria"
```

Narrator:

Intake checks specificity, duplicates, and routing
Evidence names inspected issue fields and repository paths
A stop records needs-input or blocked with a recovery owner

Expected pass artifact: structured intake comment
Expected milestone: lifecycle:triaged
Next owner: issue triage owner

Candidate narration · no gh-aw workflow executed

The Plan Ends at the Approval Block

issue #482 · candidate plan

```
## Scope
- Named files and bounded change
## Exclusions
- Work outside the issue contract
## Validation
- Exact repository commands and signals
## Rollback
- Revert the bounded pull request
## Approval
Named plan approver: @maintainer
Comment exactly /approve-plan
```

Bounded scope

Files, exclusions, validation, risk, and rollback remain inspectable

Named authority

The planned label records a milestone; it does not approve work

Consequential stop

Coding authority begins only after a fresh plan and exact authorized command

Preserve Authority Through Merge

Compare the authorized path with an expected no-op, then retain human merge acceptance.



Approve

One exact command advances



No-op

Unauthorized state stays put



Accept

CODEOWNER retains merge

Expected behavior until validation

↳ Authority changes state · requests do not

The Same Command, Different Authority

Both paths remain expected until exercised in a target repository

✗ Unauthorized requester

- 1 Posts /approve-plan
Actor lacks repository approval authority
- 2 Gate checks evidence
Plan exists but actor authority fails
- 3 Repository stays put
No partial implementation or state transition

0 Expected no-op; repository state unchanged

✓ Named maintainer

- 1 Posts /approve-plan
Actor is the named authorized maintainer
- 2 Gate checks evidence
Latest plan, command, freshness, and authority agree
- 3 Coding may begin
Only the approved plan receives implementation authority

1 Expected transition into bounded coding

Candidate behavior · target-repository validation required

Coding Returns Evidence, Not Merge Authority

candidate source declaration

```
safe-outputs:
  create-pull-request:
    title-prefix: [lifecycle]
    labels:
      - agent-generated
      - lifecycle:in-review
    draft: true
    max: 1
```

1 One bounded PR

The declared output caps creation at one draft pull request



Evidence payload

Issue, approval URL, changed files, commands, results, and owner travel together



Stop on drift

Failed checks, stale approval, unavailable environment, or scope expansion block coding

Reviewed Does Not Mean Merge Permitted

🔍 Workflow evidence

COMMENT

Reports findings without submitting APPROVE

Maps acceptance criteria to the diff

Inspects deterministic check results

Routes drift, defects, and missing evidence

🛡 Repository authority

Deterministic gates

Tests, rulesets, and security controls remain enforced

CODEOWNER or named reviewer accepts residual risk

Human can return work to coding or planning

Configured merge policy makes the final gate

Measure the Handoffs Locally

Keep stopped runs in the denominator, assign recovery owners, and pilot one handoff.



Timing

Define clocks locally



Ledger

Record stop and owner



Pilot

Validate before expansion

First adoption actions

```
↳ Compile · inspect permissions · pilot
```

Measure Handoffs with Repository Timestamps

Report medians, sample sizes, and quality companions

Intake	Issue creation to intake marker; track routing corrections	<code>created→intake</code>
Planning	Intake pass to plan marker; track revision and scope drift	<code>intake→plan</code>
Coding	Authorized approval to draft PR; track checks and deviations	<code>approve→PR</code>
Review	PR sync to review marker; track changes and escaped defects	<code>sync→review</code>

Stopped runs stay in the denominator

Evidence Before Expansion



Record recovery

For every stop, capture phase, reason, owner, resume evidence, and disposition



Compile and inspect

Compile all four sources; reject schema mismatch or undeclared generated write access



Pilot one handoff

Exercise authorized and unauthorized paths in a low-risk target repository

From Implicit Handoffs to an Inspectable Candidate Lifecycle

Before

Repository judgment mixed with deterministic automation

Transitions inferred from comments and labels

Mutation authority unclear at each phase

Failures disappear outside the workflow story

After

Four bounded sources expose phase ownership

Required evidence agrees before state advances

Approval and merge acceptance stay human-owned

Blocked state names evidence and recovery owner

4

candidate workflow sources

1

exact authorized approval command

0

runtime claims before validation

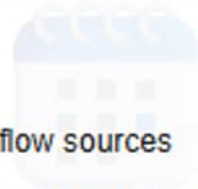
✓ WHAT YOU CAN DO TODAY

Today



- Inventory recurring repository judgments
- Name evidence and authority for each transition
- Keep issue #482 framed as a candidate walkthrough

This Week



- Compile all four workflow sources
- Inspect generated permissions and safe outputs
- Review instructions with the platform owner

This Month



- Pilot one handoff in a target repository
- Exercise authorized and unauthorized transitions
- Expand only from observed evidence

🔑 Key Takeaway

Compilation and generated-permission inspection come before any runtime claim.

GitHub Agentic Workflows

[Overview](#)

Markdown workflow model and core concepts

[How workflows work](#)

Compilation, lock files, execution, and audit markers

[Security architecture](#)

Read-only reasoning and isolated write handling

[Safe outputs](#)

Constrained repository output types and validation

Repository governance

[Copilot coding agent](#)

Repository tasks delivered through pull requests

[Actions workflow syntax](#)

Events, permissions, expressions, and execution

[CODEOWNERS](#)

File ownership and review routing



Agentic Lifecycle Orchestration

EVIDENCE-GATED HANDOFFS FROM ISSUE TO PULL REQUEST

Inspect

State, evidence, and authority must agree

Authorize

One exact approval advances a fresh plan

Recover

Blocked work retains evidence and an owner

Which single handoff should your repository compile and validate first?